



Corrotinas

Prof. Alessandro Brawerman



coroutineScope() no Android

-
- As corrotinas podem ser usadas em Activities seguindo o ciclo de vida da mesma e declarando um escopo de ciclo de vida para a própria corrotina.
 - lifecycleScope
 - Quando a Activity for destruída, o lifecycleScope das corrotinas é cancelado e todas as corrotinas nele destruídas.
 - O CoroutineContext fornece informações sobre o contexto em que a corrotina será executada.

Dispatchers

-
- Controlam em que contexto de execução as corrotinas serão executadas.
 - O contexto de execução define qual thread ou pool de threads será utilizado.
 - Isso permite que sejam executadas operações assíncronas em diferentes partes do sistema, como a thread principal da interface do usuário, operações de I/O (Input/Output), ou tarefas de processamento intenso.

Dispatchers.Main

-
- Usado na thread principal.
 - Tarefas executadas no dispatcher principal são aquelas que precisam interagir com a interface do usuário (UI).
 - Em aplicativos Android, usa-se o Dispatchers.Main para garantir que tarefas como atualizações de UI ocorram na thread correta.
 - Exemplo: atualizações de interface, navegação entre telas, manipulação de elementos visuais.
 - Onde evitar: operações demoradas ou que exigem muita CPU (para não bloquear a interface do usuário).

Dispatchers.IO

-
- Usado para operações de entrada e saída (I/O), como chamadas de API, leitura/escrita de arquivos ou comunicação com um banco de dados.
 - É otimizado para tarefas que envolvem espera, como comunicação de rede.
 - Exemplo: operações de I/O, como leitura/escrita de arquivos, chamadas de API, ou consultas de banco de dados.

Dispatchers.Default

-
- Usado para tarefas que exigem alta utilização de CPU, como cálculos complexos ou algoritmos pesados.
 - Esse dispatcher utiliza um pool de threads que é adequado para operações que envolvem processamento intensivo.
 - Exemplo: processamento de dados pesados, cálculos complexos, ou qualquer tarefa que demande um uso intenso de CPU.

Dispatchers.Unconfined

-
- Executa corrotinas na thread atual, mas pode mudar para outra thread durante a execução, dependendo de como o código suspenso é executado.
 - Geralmente não é recomendado, pois seu comportamento pode ser imprevisível.
 - Exemplo: raramente usado em produção; é mais utilizado para casos muito específicos ou testes.

Exemplo

```
fun dataRequest() {  
    // Inicia a corrotina na Main Thread  
    lifecycleScope.launch(Dispatchers.Main) {  
        progressBar.visibility = View.VISIBLE  
        // Faz a operação de I/O em segundo plano (Dispatcher.IO)  
        val result = withContext(Dispatchers.IO) {  
            // Simula a chamada de API  
            delay(2000)  
        }  
        // Atualiza a interface do usuário com o resultado  
        progressBar.visibility = View.GONE  
        textView.text = result  
    }  
}
```


Exemplo

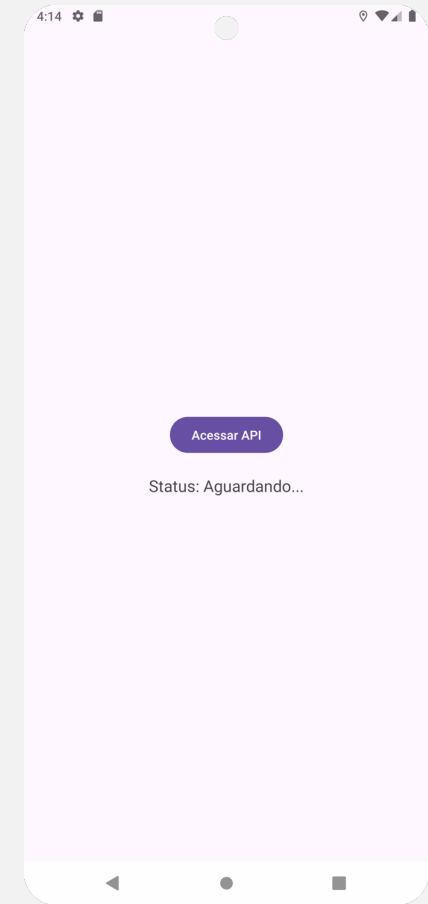
-
- UI Thread: O `launch(Dispatchers.Main)` é usado para inicializar a corrotina na thread principal, permitindo interagir com a interface do usuário.
 - I/O Thread: O `withContext(Dispatchers.IO)` é usado para mudar o contexto para um pool de threads adequado para operações de I/O (como chamadas de API ou leitura de banco de dados).
 - Troca de Contexto: O `withContext` é usado para alternar entre contextos, garantindo que a operação de I/O não bloqueie a thread principal.

App - simRequestAPI

-
- Vamos fazer um app para fixar os conceitos vistos.
 - O objetivo é simular o acesso a uma API que retornaria algum tipo de dados a ser usado na tela.
 - Crie o projeto simRequestAPI.
 - Acerte o layout conforme a imagem no próximo slide.
 - Depois acerte o controller conforme o código fornecido.

App - simRequestAPI

- Use layout linear.
- Button
 - onClick -> request
- Arraste um progressBar (na aba Widgets da paleta)
 - id – progressBar
 - visibility – gone (inicialmente invisível)
- Por fim, inclua um textView
 - id – tvStatus
 - size – 18sp
 - marginTop – 20dp



App - simRequestAPI

```
fun request(view: View) {  
    val tvStatus = findViewById<TextView>(R.id.tvStatus)  
    val progressBar = findViewById<ProgressBar>(R.id.progressBar)  
    tvStatus.text = "Status: Iniciando acesso à API..."  
    // Mostra o ProgressBar  
    progressBar.visibility = ProgressBar.VISIBLE  
  
    // Inicia a corrotina no escopo do ciclo de vida (lifecycleScope)  
    lifecycleScope.launch(Dispatchers.Main) {  
        // Simula uma chamada de API com delay de 2 segundos  
        delay(2000)  
        // Esconde o ProgressBar  
        progressBar.visibility = ProgressBar.GONE  
        // Atualiza o status após a simulação  
        tvStatus.text = "Status: Acesso à API finalizado!"  
    }  
}
```